

PhysicsMate: Curriculum-Grounded Bengali Physics QA with Small Language Models

Final Project Report · Submitted August 29, 2026

Team: Rashid Azraf · Saadman Sajid

1. Motivation

Bengali has around 230 million speakers, but it barely shows up in educational language-model research. For secondary-school science there is almost no question-answering resource tied to an actual curriculum, and the schools that would benefit most usually have weak internet and cheap hardware. We picked a concrete target: the NCTB Grade 9–10 Physics syllabus. The question we set out to answer was simple to state.

Can a small, curriculum-grounded dataset teach a small model enough Bengali physics to be useful offline on a normal laptop, and does that benefit change with model size?

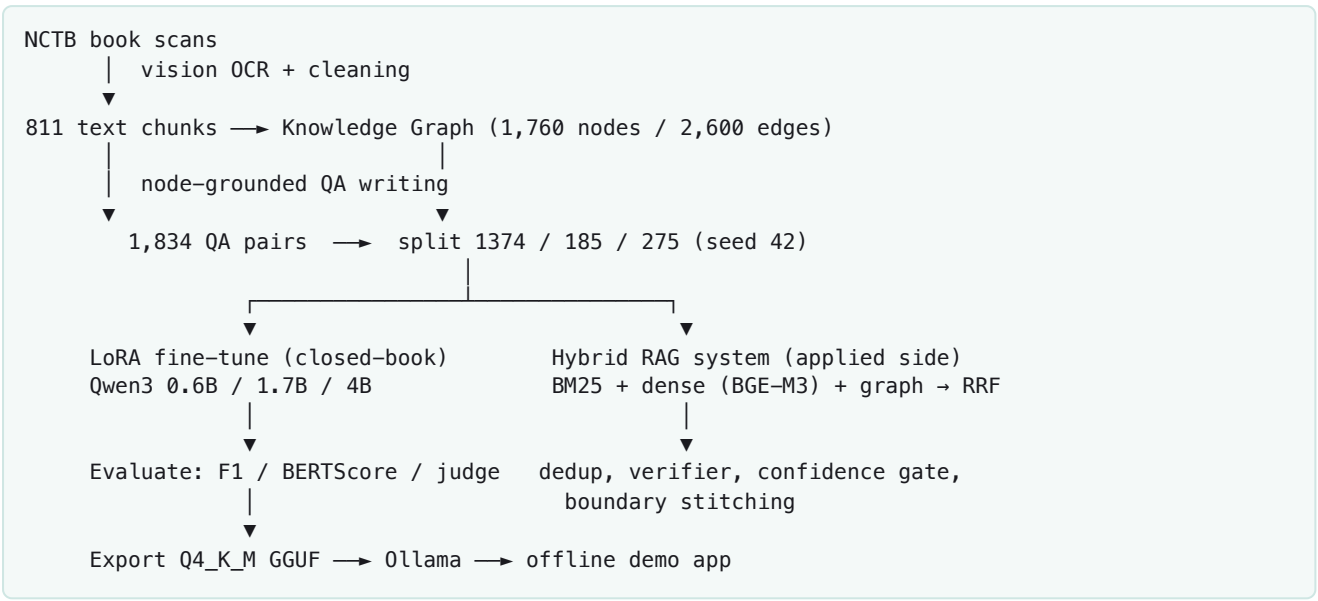
Two things made it worth doing. First, it gives us a benchmark for low-resource curriculum QA that other people can reuse. Second, it gives us something that actually runs: a fine-tuned model small enough to work with no internet, no API key, and no cloud bill. That last part matters, because that is the setting our users are in.

2. Methods (overview)

Stage	What we did
Data	Digitised the NCTB Grade 9–10 physics book (page scans, vision OCR, per-chapter Markdown, then cleaning) into 811 text chunks across 13 chapters .
Knowledge graph	Built a graph of 1,760 nodes (10 node types) and 2,600 edges to organise the domain.
QA construction	Wrote 1,834 question–answer pairs , each tied to a graph node, then split them (seed 42, stratified by chapter) into train 1,374 / val 185 / test 275 .
Adaptation	LoRA / QLoRA fine-tuning of 4-bit Qwen3 at 0.6B, 1.7B, 4B , closed-book (no retrieval), so the numbers reflect what the dataset itself puts into the weights.
Evaluation	Token-F1 and BERTScore, which are deterministic, plus an LLM-as-a-judge accuracy score, all on the held-out 275 questions.
Deployment	Exported each model to a Q4_K_M GGUF file with an Ollama Modelfile, and built an offline demo (local server to Ollama, streaming, model switcher, Bangla/English UI).

The main finding: fine-tuning helps at all three sizes, and the bigger the model, the bigger the jump (+5.5, then +15.0, then +23.3 accuracy points). Section 7 has the numbers.

3. System Overview



4. Weekly Progression

Reconstructed from file dates across the whole project (~/Downloads/transfer , ~/NCTB , ~/Desktop/conference paper). It ran about ten weeks from the first OCR script to submission.

Week	Dates (2026)	Focus	What got produced
W0	Jun 19 – Jun 30	First data pipeline (later revised)	First OCR scripts; a clean-and-chunk pass; an early chapter-level graph; the first project concept note.
W1	Jul 1 – Jul 14	First end-to-end prototype (later revised)	Dataset rebuilt a few times; QA parsed from the Udvash/SSC banks; a working retriever + agent + benchmark prototype.
W2	Jul 21 – Jul 28	Data foundation (final)	Book to Markdown, 13 chapters (Jul 21–23); figure extraction; per-chapter concept graphs; merged dataset.json (811 chunks) and graph.json (1,760 nodes / 2,600 edges) (Jul 25); 1,834 QA pairs (Jul 26–28).
W3	Aug 1 – Aug 8	Retrieval system	BM25 + dense index; hybrid retriever and fusion tuning; evaluation harness; the HSC (grade 11–12) bridge on the graph (Aug 8); first fine-tune attempts on Kaggle.
W4	Aug 9 – Aug 12	Local fine-tuning + RAG work	SFT dataset builder; local MLX LoRA fine-tune of Qwen3-4B; a 9-task RAG optimisation pass; a leakage-free "clean" dataset and an LLM judge.
W5	Aug 13 – Aug 16	Scaling study + paper	Closed-book split (1374/185/275); LoRA fine-tune of 0.6B/1.7B/4B; export to Q4_K_M GGUF + Modelfiles (Aug 13–14); deterministic evaluator, judge, and <code>verify.py</code> ; <code>results.yml</code> / <code>stats.yml</code> ; scaling figures; paper draft (Aug 16).
W6	Aug 18 – Aug 24	Deployment + write-up	Offline demo (three models through Ollama, streaming, Bangla/English) (Aug 18); UI fixes; this report and the slides (Aug 24–29).

5. Contribution of Each Team Member

Member	Main area	What they did
Rashid Azraf	Dataset OCR & model tuning	Ran the OCR that turned the Grade 9–10 book into clean text (the 811-chunk corpus); trained the LoRA/QLoRA models at 0.6B, 1.7B, and 4B; set up the closed-book training; exported the GGUF files and got them running in Ollama; ran the evaluation experiments.
Saadman Sajid	Synthetic QA, dataset analysis & writing	Wrote the 1,834 QA pairs and tied each to a graph node; did the dataset analysis (statistics, node-type and chapter breakdowns, the train/val/test design); read the related work; wrote the paper and all the reports.

Both of us worked on the knowledge graph, the demo app, final testing, and the presentation.

6. Implementation Details

(Written up, not pasted. The source is in the repository.)

6.1 Data construction

We turned the Grade 9–10 physics book into text one chapter at a time, reading each page image with a vision model and transcribing it into Markdown. We then cleaned out OCR junk and removed repeated text, which left 811 chunks.

On top of the text we built a knowledge graph. Every node has a type. The ten types and their counts are concept (397), question (252), quantity (224), figure (218), example (188), entity (176), definition (136), formula (83), law (65), and hsc_topic (21). Nodes are joined by 2,600 typed edges, for example *defines*, *depends-on*, and *illustrated-by*. Each of the 1,834 QA pairs points back to a specific node, so we can trace any answer to a place in the curriculum. We split the data once, stratified by chapter with seed 42, into train, validation, and test (1,374 / 185 / 275). The test set never appears in training.

6.2 Fine-tuning

We adapt 4-bit Qwen3 at three sizes with LoRA. LoRA trains a small set of adapter weights and leaves the base model frozen, which is what lets the whole thing fit on one consumer GPU or an Apple-Silicon machine. We used two training paths over the project: a QLoRA path (rank 16, alpha 32, dropout 0.05, adapters on the attention and MLP projections, 3 epochs, cosine schedule, 8-bit AdamW, sequence length 512, seed 42) and an Apple MLX path for the local runs. Training is closed-book. The model only ever sees a question and its answer, with no retrieved context, so whatever it learns has to live in the weights.

6.3 Evaluation

We report three numbers on the 275 held-out questions. Token-F1 and BERTScore are deterministic, and `verify.py` recomputes them and checks they match the reported table. The third number is accuracy from an LLM judge, using the MT-Bench setup, where an answer counts as correct if it carries the gold answer's key fact. The judge scored all six model variants (raw and fine-tuned at each size) under the same prompt.

6.4 Deployment

Each fine-tuned model is exported to a Q4_K_M GGUF file (0.6B is 0.40 GB, 1.7B is 1.1 GB, 4B is 2.5 GB) with an Ollama Modelfile, so it runs offline through Ollama. A small local server, written with the Python standard library and no extra packages, serves a single web page and streams tokens from Ollama. The page lets you switch between the three sizes, streams the reply as it comes, offers sample questions from the real test set, handles greetings, and toggles between Bangla and English.

6.5 Applied RAG system

Next to the closed-book study we also built a retrieval system over the same corpus: BM25 for keyword match, BGE-M3 dense embeddings for meaning, and a graph channel, combined with reciprocal rank fusion. After retrieval the context goes through near-duplicate removal, derivation-boundary stitching, a second-pass groundedness check, and a confidence gate that refuses when the top scores are too low. On a separate leakage-free retrieval subset the fine-tuned 4B model came out even with RAG, which suggests the fine-tuning can take some load off retrieval at inference time.

7. Results

Closed-book, held-out test set (n = 275). Fine-tuned (FT) against the raw base:

Model	Params	Accuracy (judge)	Token-F1	BERTScore
Qwen3-0.6B raw	0.6B	0.0%	0.216	0.696
Qwen3-0.6B FT	0.6B	5.5%	0.278	0.750
Qwen3-1.7B raw	1.7B	10.5%	0.227	0.706
Qwen3-1.7B FT	1.7B	25.5%	0.299	0.765
Qwen3-4B raw	4B	27.6%	0.252	0.722
Qwen3-4B FT	4B	50.9%	0.350	0.782

The gain gets bigger as the model gets bigger:

Model	Accuracy raw → FT	Δ Accuracy	Δ F1	Δ BERTScore
Qwen3-0.6B	0.0% → 5.5%	+5.5 pp	+6.2	+5.4
Qwen3-1.7B	10.5% → 25.5%	+15.0 pp	+7.2	+5.9
Qwen3-4B	27.6% → 50.9%	+23.3 pp	+9.8	+6.0

On deployment: the fine-tuned 4B model, the best of the three, ships as a 2.5 GB file that runs offline on a laptop, and it matches RAG on the leakage-free subset.

8. Reproducibility & Deliverables

- Reproduce the deterministic numbers with `python verify.py`. It recomputes

Token-F1 and BERTScore for all six models and checks them against `results.yml`.

- To run the models or the demo, see `submission/README.md` . It has the terminal commands to load all three models in Ollama and 50-plus test questions.
- Artefacts: the dataset and frozen split (`data/`), per-model predictions and GGUF files (`models/`), the evaluation scripts (`evaluate.py` , `grade.py` , `verify.py`), the paper draft (`paper_section_*.md`), and the demo (`demo/`).

9. Limitations & Future Work

The 0.6B model is still weak in absolute terms at 5.5%, so it is useful as a data point in the scaling story but not for a real classroom. The judge accuracy is an automatic proxy, and a human-rated subset would make it stronger. Next steps: push into HSC (grades 11–12) using the bridge already in the graph, add a human-verified test set, and try another model family such as Llama-3.2 to check the finding holds.

10. Failures, Redos, and Lessons Learned

We got to the clean result only after throwing out two full approaches. They are worth writing down, because they are the reason we trust the numbers in Section 7.

10.1 First system, rejected in review

Our first pipeline (June to early July) got built and then rejected when the faculty's research assistant looked at it.

First attempt (dropped)	What was wrong	The redo (final)
OCR with Tesseract (<code>ben+eng</code> , the whole ~366-page book in one batch, no per-page check)	Weak Bangla, garbled words and broken diacritics (you can see it in the raw <code>1.txt</code>)	A second OCR pass with a vision model, one page at a time, keeping headers and চিহ্ন X.XX figure markers, with one page checked before the full run
A "graph" of 33 nodes and 27 edges	Each node was a whole chapter, so it was a table of contents, not a graph	A concept-level graph, 1,760 nodes and 2,600 edges, 10 node types
No figures pulled from the book	The book leans on diagrams, and retrieval lost all of that	Figure extraction per chapter, one diagram at a time
3,502 noisy chunks with 85+ repeated-text bugs	Low signal, lots of duplication	Cleaned down to 811 solid chunks
QA scraped from the Udvash bank and MCQ dumps (~1,735)	Not tied to the curriculum, uneven quality	1,834 QA pairs written against graph nodes
ChromaDB + multilingual-e5 + an agentic "remediation planner"	Too broad, nothing measurable	A closed-book QA benchmark plus a model you can actually ship

The RA's notes were specific: too few graph nodes, no diagrams, needs scannable page images, extract the graphs one chapter at a time, and validate a single page before committing to a full run. We fixed all of them in the redo.

The trail is still on disk. The dropped v1 lives in `~/untitled folder/nctb-rag-project/`. Its `archive/` holds the failed early scripts, `output/` holds the dataset rebuilds (`build_dataset_v2/v3/v4.py`), and there is a literal `chunks.json.tesseract_backup.json` that marks the moment we swapped the Tesseract output for the re-OCR. The concept graph was rebuilt separately in `~/untitled folder 2/`, where the per-chapter `_enrich_chNN.py` scripts produced the `chNN_graph.json` files that merged into the final graph.

10.2 Fine-tuning that did not work at first

Even with the clean dataset, the model side failed a few times.

- The cloud GPU route fell over. About 20 Kaggle kernel pushes died on

P100/CUDA/library-version conflicts, so we moved training local (MLX / QLoRA on Apple Silicon).

- Fine-tuning on top of RAG lost to a plain baseline. Four early LoRA runs of

Qwen3-4B in the retrieval setting only matched or lost to a well-prompted, un-tuned model. That is catastrophic forgetting on a small dataset. One refusal-focused run diverged, with a validation-loss spike and broken metrics, and we stopped it.

- One evaluation run was broken. `results_tables.csv` came out with `nan`

BERTScore and duplicated 0% rows. We caught it and threw it away rather than report it.

- The RAG optimisation pass also lost. A 9-task pipeline change scored worse than

the frozen baseline on our 26-question set. We reported that honestly instead of hiding it.

10.3 What finally worked

We narrowed the scope to a clean closed-book scaling study: a strict, leakage-free split (1,374 / 185 / 275, seed 42), LoRA retrained from the clean dataset at three sizes, and evaluation with both deterministic metrics and the judge. That version was measurable, and it won cleanly. It gave us the numbers in Section 7 and the GGUF models we ship.

10.4 What we took from it

1. Tesseract is not good enough for Bangla textbook scans. Vision-model OCR was the thing that actually worked.

1. A list of chapters is not a knowledge graph. Grounding at the concept level is what made the dataset useful.

1. On a small, clean dataset, closed-book LoRA at the right size beat both fine-tuning-over-RAG and pipeline tweaking. We only knew that because we kept measuring against a frozen baseline.